

# MKU Komputer dan Pemrograman

## Minggu 9: File I/O: Olah Data CSV

Universitas Bengkulu

Ilmu Perpustakaan dan Informasi

- Memahami konsep File Input/Output (I/O) di Python.
- Mampu membuka, membaca, dan menulis file teks biasa.
- Menggunakan modul bawaan `csv` untuk mengolah data berbentuk *Comma-Separated Values*.
- Melakukan *parsing* (penguraian) data koleksi perpustakaan.
- Menerapkan proses import dan export data buku menggunakan file CSV.

# Membuka File (File I/O)

Fungsi bawaan `open()` digunakan untuk membuka file. Mode yang umum digunakan:

- `'r'` : Read (membaca file, file harus ada).
- `'w'` : Write (menulis ke file, membuat baru atau menimpa yang sudah ada).
- `'a'` : Append (menambahkan ke akhir file).

**Best Practice:** Gunakan blok `with` agar file otomatis ditutup.

```
with open('data.txt', 'r') as file:
    konten = file.read()
    print(konten)
# file otomatis ditutup di sini
```

## Menulis File ('w')

```
with open('catatan.txt', 'w') as  
    f:  
    f.write("Buku 1: Python Dasar  
            \n")  
    f.write("Buku 2: Basis Data\n")
```

## Membaca per Baris

```
with open('catatan.txt', 'r') as  
    f:  
    for baris in f:  
        print(baris.strip())
```

# Apa itu File CSV?

- CSV (*Comma-Separated Values*) adalah format file teks sederhana yang digunakan untuk menyimpan data tabular (seperti *spreadsheet* atau basis data).
- Nilai di setiap baris biasanya dipisahkan oleh koma (,), titik koma (;), atau tab.
- Sangat umum digunakan dalam manajemen data perpustakaan untuk mengeksport/mengimpor katalog buku, data anggota, dsb.

## Contoh Isi CSV:

```
id_buku,judul,pengarang,tahun  
B001,Pengantar Ilmu Perpustakaan,Sulistyo-Basuki,1991  
B002,Manajemen Koleksi,Yulia,2010
```

# Membaca File CSV: Modul csv

Python memiliki modul bawaan csv yang mempermudah pembacaan dan penulisan.

```
import csv

with open('koleksi.csv', mode='r') as file:
    # Membaca sebagai dictionary (baris pertama = header)
    csv_reader = csv.DictReader(file)

    for baris in csv_reader:
        print(baris['judul'], "-", baris['pengarang'])
```

*Keuntungan DictReader:* Memungkinkan kita memanggil kolom berdasarkan nama headernya, bukan indeks.

# Menulis File CSV

Untuk menyimpan data ke CSV, kita dapat menggunakan `csv.DictWriter`.

```
import csv

data_buku = [
    {'id': 'B003', 'judul': 'Katalogisasi', 'tahun': 2015},
    {'id': 'B004', 'judul': 'Literasi Informasi', 'tahun': 2018}
]

with open('buku_baru.csv', mode='w', newline='') as file:
    header = ['id', 'judul', 'tahun']
    writer = csv.DictWriter(file, fieldnames=header)

    writer.writeheader() # Menulis baris header
    writer.writerows(data_buku) # Menulis semua baris data
```

# Parsing Data Koleksi Perpustakaan

*Parsing* berarti mengurai data menjadi struktur yang lebih bermakna. Contoh: Kita ingin menghitung total buku yang terbit setelah tahun 2010.

```
import csv
total_buku_baru = 0

with open('koleksi.csv', 'r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        tahun_terbit = int(row['tahun']) # Parsing string ke int
        if tahun_terbit > 2010:
            total_buku_baru += 1

print(f"Jumlah buku terbitan > 2010: {total_buku_baru}")
```

# Contoh Kasus: Import dan Export Data Buku

**Skenario:** Baca data dari katalog.csv, tambahkan data buku baru, lalu simpan ke katalog\_update.csv.

```
import csv
katalog = []
# 1. Import
with open('katalog.csv', 'r') as f_in:
    reader = csv.DictReader(f_in)
    katalog = list(reader)

# 2. Tambah Data Baru
katalog.append({'id': 'B099', 'judul': 'Arsip Digital', 'tahun': '2023'})

# 3. Export
with open('katalog_update.csv', 'w', newline='') as f_out:
    writer = csv.DictWriter(f_out, fieldnames=['id', 'judul', 'tahun'])
    writer.writerow(katalog[0])
    for book in katalog[1:]:
        writer.writerow(book)
```

- Fungsi `open()` digunakan dengan mode `'r'` (baca), `'w'` (tuliskan), atau `'a'` (tambah).
- Blok `with` adalah *best practice* agar file otomatis ditutup.
- File CSV adalah standar umum untuk pertukaran data tabular.
- Gunakan modul bawaan `csv` (`csv.DictReader` dan `csv.DictWriter`) untuk menghindari penanganan pemisah (koma/titik koma) secara manual.
- *Parsing* CSV memudahkan pustakawan dalam mengotomatisasi manajemen *metadata* koleksi (import, manipulasi, export).

# Terima Kasih

Ada Pertanyaan?